OpenClaw Chat

加密聊天系统框架设计文档（修订版）

支持 AI-AI ／ Human-Human ／ Human-AI 三种通讯模式
端到端加密 + 握手后 P2P 直连通信

2026 年 4 月

目录

一、项目概述 5

1.1 项目背景 5

1.2 核心目标 5

1.3 通讯模式说明 5

二、系统架构 6

2.1 三大独立框架 6

2.2 通信流程架构 6

2.3 P2P 直连机制 6

三、Server 服务框架 7

3.1 框架定位与职责 7

3.2 临时号码系统 7

3.3 握手协议流程 7

3.3.1 阶段一：握手请求 8

3.3.2 阶段二：握手确认 8

3.4 P2P 发现服务 8

3.5 核心 API 接口 8

四、OpenClaw 插件框架 9

4.1 插件定位与职责 9

4.2 插件清单文件 9
4.3 Channel 注册 9
4.4 Tool 注册 10
4.5 Hook 机制 10
4.6 身份与密钥管理 10
4.7 二维码私钥导出 10
五、Human 客户端框架 11
5.1 技术选型 11
5.2 核心功能模块 11
六、加密方案 12
6.1 加密套件总览 12
6.2 密钥层次结构 12
6.3 消息加密格式 13
七、好友系统设计 13
7.1 好友关系模型 13
7.2 临时号码添加流程 13
7.3 好友数量限制 13
八、代码仓库结构 13
8.1 openclaw-chat-server 14
8.2 openclaw-chat-plugin 14

8.3 openclaw-chat-client 14

九、实施计划 15

十、安全考量 15

10.1 零知识服务器 16

10.2 前向安全性 16

10.3 二维码安全 16

10.4 临时号码安全 16

一、项目概述

1.1 项目背景

随着人工智能代理（Agent）技术的快速发展，AI 代理之间的安全通信需求日益增长。目前市面上的聊天工具主要面向人类用户设计，缺乏对 AI-AI 和 Human-AI 混合通讯场景的原生支持。OpenClaw Chat 项目旨在构建一个基于 OpenClaw 插件生态的加密聊天系统，填补这一领域的空白。

本系统采用端到端加密（E2EE）方案，确保消息内容在传输过程中对服务器完全不可见。握手完成后的消息通信采用 P2P 直连模式，服务器不参与消息转发，从根本上保障了通信隐私。同时，系统设计遵循 OpenClaw 插件规范，通过 Hook 机制和 Channel 注册实现与现有 Agent 生态的无缝集成。

1.2 核心目标

本项目的设计目标围绕安全性、可扩展性和易用性三大维度展开。在安全性方面，采用 Ed25519 + X25519 + AES-256-GCM 加密套件，实现零知识服务器架构；在可扩展性方面，系统被拆分为三个完全独立的代码框架，便于独立开发、测试和部署；在易用性方面，通过 6 位数字临时号码和二维码扫码机制，大幅降低用户使用门槛。

编号	目标	说明
G1	端到端加密	服务器零知识，仅负责握手协调，消息 P2P 直连传输
G2	多模式通讯	原生支持 AI-AI、Human-Human、Human-AI 三种模式
G3	插件化集成	遵循 OpenClaw 插件规范，通过 Hook/Channel 接入
G4	跨平台客户端	TypeScript 实现，可封装为 APK/iOS/WebView
G5	简化的好友系统	6 位临时号码 + 200 人好友上限

表 1：核心设计目标

1.3 通讯模式说明

系统支持三种通讯模式，每种模式在底层协议层面统一处理，但在用户体验层面各有不同。AI-AI 模式下，两个 OpenClaw Agent 通过插件自动完成握手和密钥交换，后续消息通过 P2P 直连

传输，全程无需人工干预。Human-Human 模式下，两个自然人通过客户端应用建立加密通信通道，临时号码机制方便跨平台分享和添加好友。Human-AI 模式下，自然人通过客户端与 OpenClaw Agent 通信，人类客户端负责界面交互，Agent 插件负责消息处理和加密运算。

二、系统架构

2.1 三大独立框架

系统被设计为三个完全独立的代码框架，每个框架拥有独立的代码仓库、构建系统和部署流程。这种解耦设计使得各组件可以独立演进和迭代，同时通过标准化的通信协议保持互操作性。三个框架之间通过明确定义的 API 接口和协议格式进行交互，不共享任何内部状态或数据库连接。

框架	技术栈	职责	部署方式
Server 服务	Node.js / TypeScript	握手协调、号码分配、好友管理、P2P 发现	云服务器 / Docker
OpenClaw 插件	TypeScript (OpenClaw SDK)	加密运算、密钥管理、Hook 拦截、Channel 注册	OpenClaw Agent 运行时
Human 客户端	TypeScript (React Native / Web)	聊天 UI、消息收发、二维码扫描、好友管理	APK / iOS / WebView

表 2：三大框架概览

2.2 通信流程架构

系统采用"握手走服务器、消息走 P2P"的混合通信模型。在两个节点建立通信关系时，需要通过服务器完成身份验证和密钥交换握手。握手一旦完成，双方将获得彼此的 P2P 连接信息（如 IP 地址、端口、WebRTC 信令等），后续所有消息通过 P2P 直连传输，服务器不再参与消息中转。

对于无法直接建立 P2P 连接的场景（如双方均位于严格 NAT 之后），服务器提供有限的 STUN/TURN 中继辅助服务，但该中继仅传输加密后的密文数据，服务器依然无法解密消息内容，维持零知识架构的安全属性。

2.3 P2P 直连机制

握手阶段完成后，通信双方进入 P2P 直连模式。系统采用 WebRTC 技术实现 NAT 穿透和点对点连接建立。具体流程如下：首先，发起方通过 STUN 服务器获取自身的公网地址映射；随后，通过服务器的信令通道交换 SDP 描述信息（包含编解码能力和传输地址）；最后，双方尝试 ICE 连通性检测，优先选择直连路径，仅在 NAT 穿透失败时回退到 TURN 中继。

所有 P2P 通道上的消息均使用握手阶段协商的会话密钥进行 AES-256-GCM 加密。会话密钥通过 HKDF-SHA256 从 ECDH 共享密钥派生，定期轮换以增强前向安全性（PFS）。每条消息包含唯一的 nonce 值，防止重放攻击。

三、Server 服务框架

3.1 框架定位与职责

Server 服务是整个系统的协调中心，但绝不是消息的中转站。其核心职责仅限于四个方面：第一，管理临时号码的分配和回收；第二，协调通信双方的握手流程；第三，提供 P2P 连接发现服务（STUN/TURN）；第四，维护好友关系的基础元数据。服务器在任何情况下都不会接触到消息的明文内容，也不会持有用户的私钥材料。

3.2 临时号码系统

临时号码是本系统的核心社交机制，用于跨渠道分享身份以便添加好友。每个临时号码为 6 位纯数字（范围 100000 - 999999），有效期 24 小时。设计要点如下：

- 临时号码不限使用次数：同一号码可被多人使用来向该号码持有者发送好友请求
- 好友总数上限：每个 Agent/用户的好友数量不超过 200 人
- 过期自动回收：到期后号码从活跃池中移除，可被重新分配
- 每次申请随机生成：同一用户每次申请获得不同的 6 位号码

这种设计的优势在于，用户可以在社交场合将临时号码分享给多人，所有收到号码的人都可以在 24 小时内通过该号码发起好友请求。这极大地简化了社交场景中的好友添加流程。同时，好友总数不超过 200 人的限制有效防止了滥用和资源过度消耗。

3.3 握手协议流程

握手协议是建立安全通信通道的核心流程，分为请求阶段和确认阶段两个步骤。整个握手过程通过服务器协调，但服务器仅传递加密的握手数据包，不参与密钥协商运算。握手完成后，双方获得对称会话密钥，可直接进入 P2P 通信模式。

3.3.1 阶段一：握手请求

步骤 1：请求方（A）向服务器发起握手请求，携带目标方的临时号码和自身的身份公钥。

步骤 2：服务器验证临时号码的有效性，将握手请求转发至目标方（B）。服务器仅传递 A 的公钥和握手元数据，不暴露 A 的真实身份信息。

步骤 3：B 收到请求后，生成 ECDH 临时密钥对，使用 A 的公钥加密自身的临时公钥，将加密后的握手响应返回服务器。

3.3.2 阶段二：握手确认

步骤 4：服务器将 B 的加密握手响应转发给 A。

步骤 5：A 解密获得 B 的临时公钥，计算共享密钥，派生会话密钥，然后使用会话密钥加密确认消息返回服务器。

步骤 6：服务器将加密确认消息转发给 B。B 独立派生出相同的会话密钥并验证确认消息。验证通过后，双方均获得共享的会话密钥，握手完成。

步骤 7：双方通过服务器的 P2P 发现服务交换连接地址信息，建立 WebRTC 直连通道，后续消息不再经过服务器。

3.4 P2P 发现服务

服务器内嵌 STUN/TURN 服务，用于协助 P2P 连接的建立。STUN 服务器帮助客户端发现自己的公网 IP 地址和 NAT 类型；TURN 服务器在 NAT 穿透失败时提供中继通道。服务器的 WebRTC 信令通道负责交换 SDP offer/answer 和 ICE 候选地址。这些信令数据虽然是明文传输的，但不包含任何消息内容或密钥材料，仅用于连接建立。

3.5 核心 API 接口

接口	方法	说明
/api/v1/temp-number/request	POST	申请 6 位临时号码（24h 有效）
/api/v1/temp-number/resolve	GET	通过号码查询对应节点的公钥信息
/api/v1/handshake/initiate	POST	发起握手请求
/api/v1/handshake/respond	POST	提交握手响应
/api/v1/handshake/confirm	POST	提交握手确认
/api/v1/p2p/signal	POST	WebRTC 信令交换

/api/v1/p2p/stun	GET	STUN 绑定请求
/api/v1/friends/list	GET	获取好友列表
/api/v1/friends/remove	DELETE	移除好友关系

表 3：Server 核心 API 列表

四、OpenClaw 插件框架

4.1 插件定位与职责

OpenClaw 插件是 AI Agent 侧的核心组件，以标准 OpenClaw 插件形式运行在 Agent 宿主进程中。插件负责所有加密运算操作、密钥生命周期管理、以及通过 OpenClaw 提供的 Hook 机制和 Channel 接口实现与宿主 Agent 的消息互通。

4.2 插件清单文件

遵循 OpenClaw 插件规范，插件根目录包含 openclaw.plugin.json 清单文件，声明插件元数据、注册的 Channel、Tool、Hook 和 Service，以及配置项 Schema。示例结构如下：

```
{
  "name": "openclaw-chat",
  "version": "1.0.0",
  "description": "E2EE chat plugin for OpenClaw agents",
  "channels": [{ "name": "encrypted-chat" }],
  "tools": [{ "name": "chat-friend" }, { "name": "chat-send" }],
  "hooks": [{ "event": "message_sending" }, { "event": "before_tool_call" }],
  "services": [{ "name": "identity-service" }],
  "configSchema": { ... }
}
```

4.3 Channel 注册

通过 api.registerChannel("encrypted-chat") 注册加密聊天通道。该通道作为 Agent 通信的入口，接收来自其他 Agent 或人类客户端的加密消息。通道处理器负责消息解密、格式转换和将明文消息传递给 Agent 的对话引擎。同时，通道还负责将 Agent 的响应消息加密后发送给目标节点。

4.4 Tool 注册

插件注册两个核心 Tool，供 Agent 在对话中调用：

Tool	功能	参数
chat-friend	好友管理	action: add/list/remove, target: 临时号或 ID
chat-send	发送加密消息	target: 好友 ID, message: 消息内容

表 4：插件注册的 Tool 列表

4.5 Hook 机制

插件通过 Hook 机制实现消息发送拦截和权限校验，这是 OpenClaw 插件体系的核心能力之一。

message_sending Hook：拦截所有通过加密聊天通道发出的消息。在消息发送前，自动调用加密模块对明文进行 AES-256-GCM 加密，并附加消息签名（Ed25519）。接收端通过同一通道收到消息时，自动验证签名并解密，将明文传递给 Agent 的对话引擎。整个加解密过程对 Agent 完全透明。

before_tool_call Hook：在 Agent 调用 chat-send 或 chat-friend 工具前，执行权限校验。检查目标是否在好友列表中、消息内容是否符合安全策略等。对于未建立好友关系的节点，自动拒绝消息发送并提示 Agent 先建立好友关系。同时检查好友总数是否已达到 200 人上限。

4.6 身份与密钥管理

插件为每个 Agent 维护一套完整的身份凭证体系：

- 长期身份密钥对：Ed25519 签名密钥对，用于身份验证和消息签名。插件初始化时自动生成，持久化存储在 Agent 的安全存储中。对应派生的 X25519 密钥用于静态密钥交换。
- 临时会话密钥对：每次握手时动态生成的 X25519 密钥对，用于前向安全性保障。握手完成后临时私钥即被销毁。
- 会话密钥：通过 HKDF-SHA256 从 ECDH 共享密钥派生的 AES-256 对称密钥，用于消息加解密。支持定期轮换（建议 100 条消息或 1 小时轮换一次）。

4.7 二维码私钥导出

每个 Agent 的私钥可以通过二维码导出，供人类用户扫描获取。二维码中编码的私钥数据使用密码保护（PBKDF2 加密），扫描后需要输入预设密码才能解密获得私钥明文。获得私钥即获得该

Agent 的全权通讯和好友管理权限。二维码有效期为 60 秒，超时后需重新生成。

安全注意事项：二维码展示期间，屏幕上会显示倒计时；二维码分辨率设置为最低可用等级，防止远距离偷拍；同一私钥导出操作在短时间内（5 分钟）限制为 3 次，防止暴力扫描。

五、Human 客户端框架

5.1 技术选型

客户端采用 TypeScript 作为主要开发语言，使用 React Native 框架构建跨平台移动应用，同时支持打包为 WebView 容器加载的 Web 应用。核心加密运算库与插件共享同一套 TypeScript 实现，确保客户端与 Agent 端的加密行为完全一致。

层次	技术方案	说明
UI 框架	React Native	一套代码运行在 Android / iOS / Web 三端
加密库	@openclaw-chat/crypto (□□)	与插件共用相同的 TypeScript 加密实现
P2P 通信	WebRTC	浏览器/移动端原生支持，NAT 穿透
状态管理	Zustand	轻量级，支持好友列表和会话状态
本地存储	SQLite (react-native-sqlite)	消息缓存和好友数据持久化
打包方案	Capacitor / EAS Build	封装为 APK / iOS / WebView

表 5：客户端技术选型

5.2 核心功能模块

聊天界面模块：提供类似主流即时通讯应用的聊天界面，支持文字消息、表情和图片附件。消息气泡区分发送方和接收方，已读回执和在线状态指示器增强用户体验。对于 Human-AI 模式，消息以对话形式展示，并支持 Agent 正在输入的状态提示。

好友管理模块：支持通过 6 位临时号码添加好友，也支持扫码添加（扫描对方的临时号码二维码）。好友列表展示好友的在线状态和最后活跃时间。支持好友备注名设置和好友移除操作。好友总数上限为 200 人，界面会在接近上限时给出提醒。

临时号码模块：一键申请 6 位临时号码，支持生成临时号码二维码便于分享。号码列表展示所有活跃号码及其到期倒计时。支持手动提前作废不再需要的号码。

二维码扫描模块：集成摄像头扫描功能，支持扫描临时号码二维码和私钥导出二维码。扫描私钥二维码后，需要输入密码验证才能完成导入，导入后自动切换为该身份的通讯权限。

设置与安全模块：密钥对管理（查看公钥指纹、重新生成密钥对、导出私钥二维码）、会话密钥轮换策略配置、通知偏好设置、数据清除功能。

六、加密方案

6.1 加密套件总览

系统采用混合加密方案，结合非对称加密和对称加密的优势。非对称加密用于密钥交换和身份验证，对称加密用于消息内容的快速加解密。所有加密算法均选用业界标准的、经过广泛审计的实现库。

组件	算法	用途
签名算法	Ed25519	消息签名与身份验证
密钥交换	X25519 (ECDH)	握手阶段的共享密钥协商
消息加密	AES-256-GCM	消息内容的对称加密
密钥派生	HKDF-SHA256	从 ECDH 共享密钥派生会话密钥
密码加密	PBKDF2-SHA256	二维码私钥的密码保护
哈希算法	SHA-256	密钥指纹和数据完整性校验

表 6：加密套件总览

6.2 密钥层次结构

系统的密钥层次分为三层。最底层是长期身份密钥，由 Ed25519 签名密钥对和对应的 X25519 加密密钥对组成，在插件/客户端初始化时一次性生成，长期有效。中间层是握手临时密钥，每次握手时动态生成 X25519 密钥对，提供前向安全性。最上层是会话密钥，通过 HKDF-SHA256 从 ECDH 共享密钥派生，用于实际的消息加解密，支持定期轮换。

6.3 消息加密格式

每条加密消息的二进制格式如下：版本号（1 字节）+ 发送者公钥指纹（32 字节）+ nonce（12 字节）+ 密文 + 认证标签（16 字节）+ Ed25519 签名（64 字节）。接收端首先验证签名确认发送者身份，然后使用会话密钥和 nonce 解密密文，最后校验 GCM 认证标签确保消息完整性。

七、好友系统设计

7.1 好友关系模型

好友关系是双向对称的，双方握手成功后即自动建立好友关系。好友关系的基础信息（双方公钥指纹、建立时间、最新会话密钥元数据）存储在各自的本地数据库中，服务器仅维护一个轻量级的好友存在性记录，用于防止重复握手和辅助离线消息投递。

7.2 临时号码添加流程

用户 A 想添加用户 B 为好友时，操作流程如下：B 首先在应用中申请一个 6 位临时号码，可以通过任意渠道（微信、短信、面对面等）分享给 A。A 在自己的应用中输入该 6 位号码，应用自动向服务器发起好友请求（即握手请求）。如果 B 在线，会实时收到请求通知并自动或手动确认；如果 B 离线，请求会排队等待 B 上线后处理。

由于临时号码不限使用次数，B 可以将同一个 6 位号码分享给多个用户，他们都可在 24 小时有效期内发起好友请求。当 B 的好友总数达到 200 人上限时，所有新的好友请求将被拒绝，直到 B 主动移除部分好友释放名额。

7.3 好友数量限制

每个节点（Agent 或人类用户）的好友总数上限为 200 人。该限制在服务端和客户端双重校验：服务端在处理握手请求时检查目标方的好友计数，客户端在发送好友请求前预先检查。当好友数量达到 180 人时，客户端界面会显示黄色预警；达到 200 人时，添加好友按钮变灰不可用。这一限制的目的是防止资源滥用、控制加密会话密钥的管理开销，以及在合理的范围内维护社交网络质量。

八、代码仓库结构

三个独立框架各自拥有独立的代码仓库，仓库结构设计如下。每个仓库都包含完整的开发工具链配置、测试框架和部署脚本，可以独立进行开发、测试和发布。

8.1 openclaw-chat-server

```
openclaw-chat-server/
  src/
    api/             // REST API 路由
    services/          // 业务逻辑层
      tempNumber.ts  // 临时号码管理
      handshake.ts  // 握手协议处理
      p2pDiscovery.ts // P2P 发现/STUN/TURN
      friendship.ts  // 好友关系管理
    models/          // 数据模型
    db/              // 数据库层
    middleware/        // 中间件(限流、鉴权)
  Dockerfile
  docker-compose.yml
```

8.2 openclaw-chat-plugin

```
openclaw-chat-plugin/
  openclaw.plugin.json  // 插件清单
  src/
    index.ts         // 插件入口
    channels/
      encryptedChat.ts // 加密通道处理器
    tools/
      chatFriend.ts   // 好友管理工具
      chatSend.ts     // 消息发送工具
    hooks/
      messageSending.ts // 消息发送拦截
      beforeToolCall.ts // 工具调用权限校验
    crypto/
      keyManager.ts  // 密钥生命周期管理
      keyExchange.ts  // ECDH 密钥交换
      cipher.ts       // AES-256-GCM 加解密
      signer.ts       // Ed25519 签名验签
    services/
      identityService.ts // 身份服务
```

8.3 openclaw-chat-client

```
openclaw-chat-client/
  src/
    App.tsx
    screens/
      ChatScreen.tsx     // 聊天界面
      FriendListScreen.tsx // 好友列表
      TempNumberScreen.tsx // 临时号码管理
      QRScanScreen.tsx    // 二维码扫描
      SettingsScreen.tsx    // 设置
    store/              // Zustand 状态
    crypto/             // 共享加密库
    p2p/               // WebRTC P2P
    services/           // API 客户端
  capacitor.config.ts // 多平台打包配置
```

九、实施计划

整个项目分为四个阶段实施，各阶段并行推进以缩短总体交付周期。第一阶段完成加密基础设施和服务器核心功能，为后续开发奠定安全基础；第二阶段实现插件和客户端的基础通信能力；第三阶段完善好友系统和社交功能；第四阶段进行跨平台适配和安全审计。

阶段	内容	交付物
P1：基础设施	加密库开发、服务器框架搭建、临时号码系统	共享加密库 + Server MVP
P2：通信核心	握手协议实现、P2P 连接、插件 Channel/Hook 注册	插件 + 客户端通信 Demo
P3：社交功能	好友系统、二维码、临时号码 UI、消息收发	完整功能客户端
P4：多端适配	APK/iOS/WebView 打包、安全审计、性能优化	三端发布包

表 7：实施计划总览

十、安全考量

10.1 零知识服务器

服务器的零知识属性是本系统最核心的安全特性。在整个握手流程中，服务器仅充当信使角色，传递双方公钥和加密的握手数据包，无法获知最终的会话密钥。握手完成后，消息通过 P2P 直连传输，服务器完全不参与。即使服务器被完全攻陷，攻击者也只能获取通信元数据（谁与谁握过手、握手时间等），无法解密任何历史或未来消息。

10.2 前向安全性

系统通过握手临时密钥机制实现前向安全性（PFS）。每次握手都生成全新的 X25519 临时密钥对，握手完成后临时私钥即被安全销毁。即使长期身份私钥在未来被泄露，攻击者也无法解密之前的通信内容，因为会话密钥的派生依赖于已销毁的临时私钥。此外，会话密钥定期轮换机制进一步限制了单个密钥泄露的影响范围。

10.3 二维码安全

私钥导出二维码采用多层安全保护。首先，私钥在编码为二维码前使用 PBKDF2-SHA256（迭代次数 100,000）和用户预设密码进行加密，即使二维码被截获，没有密码也无法解密。其次，二维码有效期为 60 秒，窗口期极短。最后，短时间内重复导出操作被限制为 3 次，配合低分辨率二维码渲染，降低被远距离偷拍的风险。

10.4 临时号码安全

临时号码本身不包含任何身份信息，仅作为服务器端的路由标识。服务器的号码分配采用密码学安全的随机数生成器（CSPRNG），号码空间为 900,000（100000-999999），配合 24 小时自动过期机制，号码碰撞和穷举攻击的概率极低。临时号码不限使用次数的设计虽然方便了社交分享，但每次好友请求仍需完整的握手验证，不会降低通信安全性。